

§ 10.2 堆

基于列表或向量等结构的实现方式，之所以无法同时保证`insert()`和`delMax()`操作的高效率，原因在于其对优先级的理解过于机械，以致始终都保存了全体词条之间的全序关系。实际上，尽管优先级队列的确隐含了“所有词条可相互比较”这一条件，但从操作接口层面来看，并不需要真正地维护全序关系。比如执行`delMax()`操作时，只要能够确定全局优先级最高的词条即可；至于次高者、第三高者等其余词条，目前暂时不必关心。

有限偏序集的极值必然存在，故此时借助堆（heap）结构维护一个偏序关系即足矣。堆有多种实现形式，以下首先介绍其中最基本的一种形式——完全二叉堆（complete binary heap）。

10.2.1 完全二叉堆

■ 结构性与堆序性

如图10.1实例所示，完全二叉堆应满足两个条件。

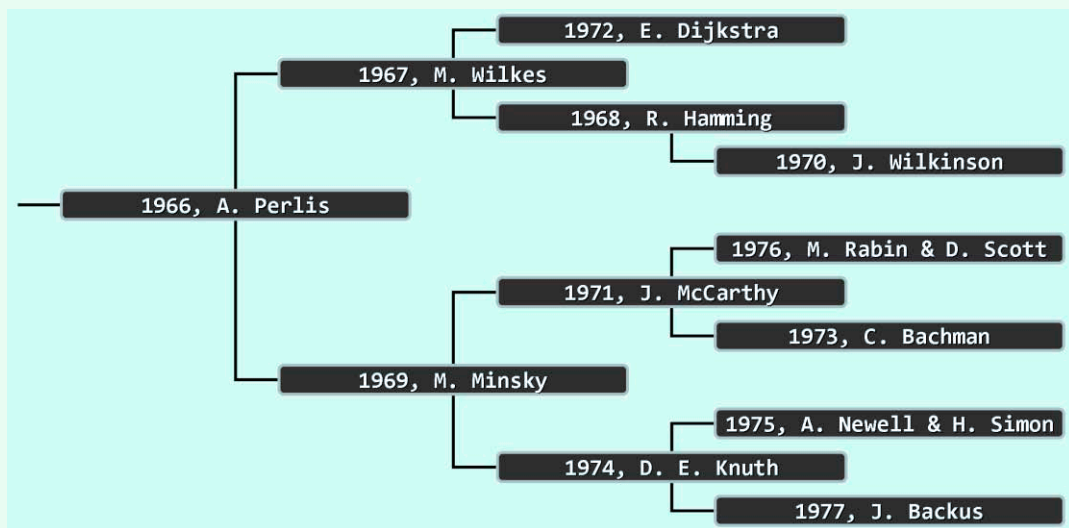


图10.1 以获奖先后为优先级，由前12届图灵奖得主构成的完全二叉堆

首先，其逻辑结构须等同于完全二叉树，此即所谓的“结构性”。如此，堆节点将与词条一一对应，故此后凡不致引起误解时，我们将不再严格区分“堆节点”与“词条”。其次，就优先级而言，堆顶以外的每个节点都不高（大）于其父节点，此即所谓的“堆序性”。

■ 大顶堆与小顶堆

由堆序性不难看出，堆中优先级最高的词条必然始终处于堆顶位置。因此，堆结构的`getMax()`操作总是可以在 $O(1)$ 时间内完成。

堆序性也可对称地约定为“堆顶以外的每个节点都不低（小）于其父节点”，此时同理，优先级最低的词条，必然始终处于堆顶位置。为以示区别，通常称前（后）者为大（小）顶堆。

小顶堆和大顶堆是相对的，而且可以相互转换。实际上，我们不久之前刚刚见过这样的实例——在代码5.29中重载Huffman超字符的比较操作符时，通过对超字符权重取负，颠倒优先级关系，使之与算法的实际语义及需求相吻合。

■ 高度

结构等同于完全二叉树的堆，必然不致太高。具体地，由5.5.2节的分析结论， n 个词条组成的堆的高度 $h = \lfloor \log_2 n \rfloor = O(\log n)$ 。稍后我们即将看到，`insert()`和`delMax()`操作的复杂度将线性正比于堆的高度 h ，故它们均可在 $O(\log n)$ 的时间内完成。

■ 基于向量的紧凑表示

尽管二叉树不属于线性结构，但作为其特例的完全二叉树，却与向量有着紧密的对应关系。

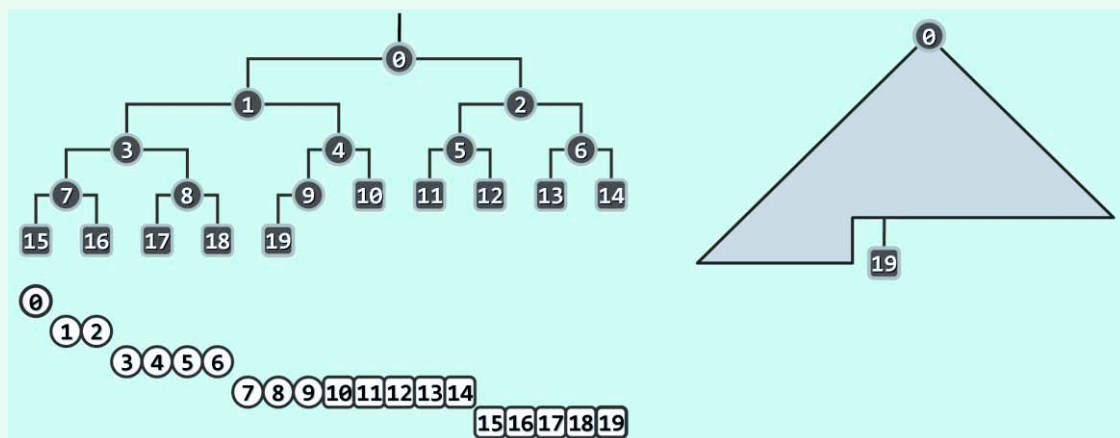


图10.2 按照层次遍历序列，对完全二叉树节点做编号（其中圆形表示内部节点，方形表示外部节点）

由图10.2可见，完全二叉堆的拓扑联接结构，完全由其规模 n 确定。按照层次遍历的次序，每个节点都对应于唯一的编号；反之亦然。故若将所有节点组织为一个向量，则堆中各节点（编号）与向量各单元（秩）也将彼此一一对应！

这一实现方式的优势首先体现在，各节点在物理上连续排列，故总共仅需 $O(n)$ 空间。而更重要地是，利用各节点的编号（或秩），也可便捷地判别父子关系。

具体地，若将节点 v 的编号（秩）记作 $i(v)$ ，则根节点及其后代节点的编号分别为：

```
i(root) = 0
i(lchild(root)) = 1
i(rchild(root)) = 2
i(lchild(lchild(root))) = 3
...
```

更一般地，不难验证，完全二叉堆中的任意节点 v ，必然满足：

```
1) 若v有左孩子，则i(lchild(v)) = 2 · i(v) + 1;
2) 若v有右孩子，则i(rchild(v)) = 2 · i(v) + 2;
3) 若v有父节点，则i(parent(v)) = ⌊(i(v) - 1)/2⌋ = ⌈(i(v)/2)⌉ - 1
```

最后，由于向量支持低分摊成本的扩容调整，故随着堆的规模和内容不断地动态调整，除标准接口以外的操作所需的时间可以忽略不计。

所有这些良好的性质，不仅为以下基于向量实现堆结构提供了充足的理由，同时也从基本的原理和方法的层面提供了有力的支持。

■ 宏

为简化后续算法的描述及实现，可如代码10.3所示预先设置一系列的宏定义。

```

1 #define InHeap(n, i)      (((-1) < (i)) && ((i) < (n))) //判断PQ[i]是否合法节点
2 #define Parent(i)        ((i-1) >> 1) //PQ[i]的父节点 ( floor((i-1)/2), i无论正负 )
3 #define LastInternal(n)   Parent(n-1) //最后一个内部节点 ( 即末节点的父亲 )
4 #define LChild(i)         (1 + ((i) << 1)) //PQ[i]的左孩子
5 #define RChild(i)         ((1 + (i)) << 1) //PQ[i]的右孩子
6 #define ParentValid(i)    (0 < i) //判断PQ[i]是否有父亲
7 #define LChildValid(n, i) InHeap(n, LChild(i)) //判断PQ[i]是否有一个 ( 左 ) 孩子
8 #define RChildValid(n, i) InHeap(n, RChild(i)) //判断PQ[i]是否有两个孩子
9 #define Bigger(PQ, i, j)  (lt(PQ[i], PQ[j]) ? j : i) //取大者 ( 等时前者优先 )
10 #define ProperParent(PQ, n, i) \
11     (RChildValid(n, i) ? Bigger(PQ, Bigger(PQ, i, LChild(i)), RChild(i)) : \
12     (LChildValid(n, i) ? Bigger(PQ, i, LChild(i)) : i \
13     ) \
14     ) //父子 ( 至多 ) 三者中的大者 ( 相等时父节点优先, 如此可避免不必要的交换 )

```

代码10.3 为简化完全二叉堆算法的描述及实现而定义的宏

■ PQ_ComplHeap模板类

按照以上思路，可以借助多重继承的机制，定义完全二叉堆模板类如代码10.4所示。

```

1 #include "../PQ/PQ.h" //引入优先级队列ADT
2 #include "../Vector/Vector.h" //引入向量
3 template <typename T>
4 class PQ_ComplHeap : public PQ<T>, public Vector<T> { //基于向量，以完全二叉堆形式实现的PQ
5 protected:
6     Rank percolateDown(Rank n, Rank i); //下滤
7     Rank percolateUp(Rank i); //上滤
8     void heapify(Rank n); //Floyd建堆算法
9 public:
10     PQ_ComplHeap() { } //默认构造
11     PQ_ComplHeap(T* A, Rank lo, Rank hi) { copyFrom(A, lo, hi); heapify(hi - lo); } //批量构造
12     PQ_ComplHeap(T* A, Rank n) { copyFrom(A, 0, n); heapify(n); } //批量构造
13     void insert(T); //按照比较器确定的优先级次序插入词条
14     T getMax(); //取出优先级最高的词条
15     T delMax(); //删除优先级最高的词条
16 }; //PQ_ComplHeap

```

代码10.4 完全二叉堆接口

■ getMax()

既然全局优先级最高的词条总是位于堆顶，故如代码10.5所示，只需返回向量的首单元，即可在 $O(1)$ 时间内完成getMax()操作。

```

1 template <typename T> T PQ_ComplHeap<T>::getMax() //获取非空完全二叉堆中优先级最高的词条
2 { return _elem[0]; } //按照此处约定, 向量首词条即优先级最高的词条

```

代码10.5 完全二叉堆getMax()接口

10.2.2 元素插入

本节介绍插入操作insert(e)的实现。因堆中的节点与其中所存词条以及词条的关键码完全对应, 故沿用此前的习惯, 在不致歧义的前提下, 以下对它们将不再严格区分。

■ 算法

如代码10.6所示, 插入算法分为两个步骤。

```

1 template <typename T> void PQ_ComplHeap<T>::insert(T e) { //将词条插入完全二叉堆中
2     Vector<T>::insert(e); //首先将新词条接至向量末尾
3     percolateUp(_size - 1); //再对该词条实施上滤调整
4 }

```

代码10.6 完全二叉堆insert()接口的主体框架

首先, 调用向量的标准插入接口, 将新词条接至向量的末尾。得益于向量结构良好的封装性, 这里无需关心这一步骤的具体细节, 尤其是无需考虑溢出扩容等特殊情况。

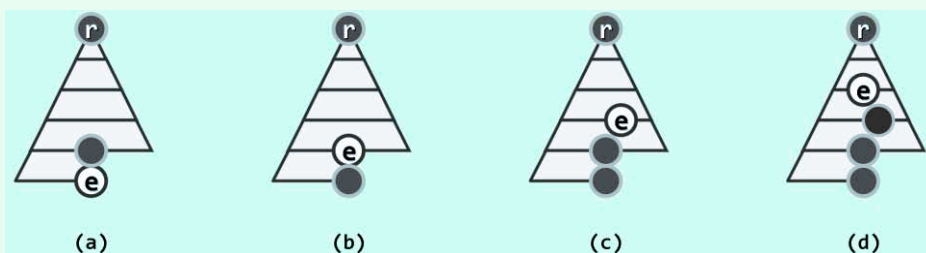


图10.3 完全二叉堆词条插入过程

尽管此时如图10.3(a)所示, 新词条的引入并未破坏堆的结构性, 但只要新词条e不是堆顶, 就有可能与其父亲违反堆序性。

当然, 其它位置的堆序性依然满足。故以下将调用percolateUp()函数, 对新接入的词条做适当调整, 在保持结构性的前提下恢复整体的堆序性。

■ 上滤

不妨假定原堆非空, 于是新词条e的父亲p(深色节点)必然存在。根据e在向量中对应的秩, 可以简便地确定词条p对应的秩, 即 $i(p) = \lfloor (i(e) - 1) / 2 \rfloor$ 。

此时, 若经比较判定 $e \leq p$, 则堆序性在此局部以至全堆均已满足, 插入操作因此即告完成。反之, 若 $e > p$, 则可在向量中令e和p互换位置。如图10.3(b)所示, 如此不仅全堆的结构性依然满足, 而且e和p之间的堆序性也得以恢复。

当然, 此后e与其新的父亲, 可能再次违背堆序性。若果真如此, 不妨继续套用以上方法, 如图10.3(c)所示令二者交换位置。当然, 只要有必要, 此后可以不断重复这种交换操作。

每交换一次, 新词条e都向上攀升一层, 故这一过程也形象地称作上滤(percolate up)。当然, e至多上滤之堆顶。一档上滤完成, 则如图10.3(d)所示, 全堆的堆序性必将恢复。

由上可见，上滤调整过程中交换操作的累计次数，不致超过全堆的高度 $\lfloor \log_2 n \rfloor$ 。而在向量中，每次交换操作只需常数时间，故上滤调整乃至整个词条插入算法整体的时间复杂度，均为 $O(\log n)$ 。这也是从一个方面，兑现了10.1节末尾就优先级队列性能所做的承诺。

当然，不难通过构造实例说明，新词条有时确实需要一直上滤至堆顶。然而实际上，此类最坏情况通常极为罕见。以常规的随机分布而言，新词条平均需要爬升的高度，要远远低于直觉的估计（习题[10-6]）。在此类场合中，优先级队列相对于其它数据结构的性能优势，也因这一特性得到了进一步的巩固。

■ 实例

通过上滤调整实现插入操作的一个实例，如图10.4所示。图中上方为完全堆的拓扑联接结构，下方为物理上与之对应的线性存储结构。

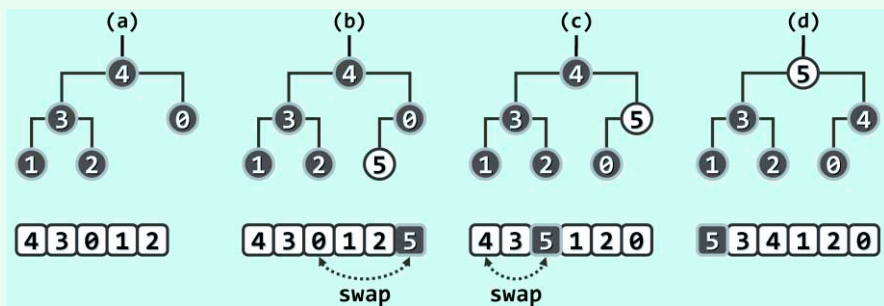


图10.4 完全二叉堆词条插入操作实例

在如图(a)所示由5个元素组成的初始完全堆中，现拟插入关键码为5的新元素。为此，首先如图(b)所示，将该元素置于向量的末尾。此时，新元素5与其父节点0逆序，故如图(c)所示，经一次交换之后，新元素5上升一层。此后，新元素5与其新的父节点4依然逆序，故如图(d)所示，经一次交换后再上升一层。此时因已抵达堆顶，插入操作完毕，故算法终止。

■ 实现

以上调整在向量中的具体操作过程，可描述和实现如代码10.7所示。

```
1 //对向量中的第i个词条实施上滤操作, i < _size
2 template <typename T> Rank PQ_ComplHeap<T>::percolateUp(Rank i) {
3     while (ParentValid(i)) { //只要i有父亲(尚未抵达堆顶)
4         Rank j = Parent(i); //j为i之父
5         if (lt(_elem[i], _elem[j])) break; //一旦当前父子不再逆序, 上滤旋即完成
6         swap(_elem[i], _elem[j]); i = j; //否则, 父子交换位置, 并继续考查上一层
7     } //while
8     return i; //返回上滤最终抵达的位置
9 }
```

代码10.7 完全二叉堆的上滤

其中为简化描述而使用的快捷方式ParentValid(), 以宏的形式定义如代码10.3所示。需说明的是，若仅考虑插入操作，则因被调整词条的秩总是起始于 $n - 1$ ，故无需显式地指

定输入参数*i*。然而，考虑到上滤调整可能作为一项基本操作用于其它场合（习题[10-12]），届时被调整词条的秩可能任意，故为保持通用性，这里不妨保留一项参数以指定具体的起始位置。

■ 改进

在如代码10.7所示的版本中，最坏情况下在每一层次都要调用一次`swap()`，该操作通常包含三次赋值。而若能注意到，参与这些操作的词条之间具有很强的相关性，则不难改进为平均每层大致只需一次赋值（习题[10-3]），而大小比较操作甚至可以更少（习题[10-4]）。

10.2.3 元素删除

■ 算法

下面再来讨论`delMax()`方法的实现。如代码10.8所示，删除算法也分为两个步骤。

```
1 template <typename T> T PQ_ComplHeap<T>::delMax() { //删除非空完全二叉堆中优先级最高的词条
2     T maxElem = _elem[0]; _elem[0] = remove(_size - 1); //摘除堆顶（首词条），代之以末词条
3     percolateDown(_size, 0); //对新堆顶实施下滤调整
4     return maxElem; //返回此前备份的最大词条
5 }
```

代码10.8 完全二叉堆`delMax()`接口的主体框架

首先，既然待删除词条*r*总是位于堆顶，故可直接将其取出并备份。此时如图10.5(a)所示，堆的结构性将被破坏。为修复这一缺陷，可如图(b)所示，将最末尾的词条*e*转移至堆顶。

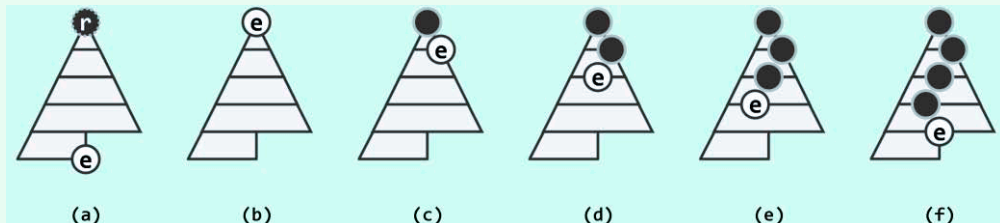


图10.5 完全二叉堆词条删除过程

当然，新的堆顶可能与其孩子（们）违背堆序性——尽管其它位置的堆序性依然满足。故以下调用`percolateDown()`函数调整新堆顶，在保持结构性的前提下，恢复整体的堆序性。

■ 下滤

若新堆顶*e*不满足堆序性，则可如图10.5(c)所示，将*e*与其（至多）两个孩子中的大者（图中深色节点）交换位置。与上滤一样，由于使用了向量来实现堆，根据词条*e*的秩可便捷地确定其孩子的秩。此后，堆中可能的缺陷依然只能来自于词条*e*——它与新孩子可能再次违背堆序性。若果真如此，不妨继续套用以上方法，将*e*与新孩子中的大者交换，结果如图(d)所示。实际上，只要有必要，此后可如图(e)和(f)不断重复这种交换操作。

因每经过一次交换，词条*e*都会下降一层，故这一调整过程也称作下滤（`percolate down`）。与上滤同理，这一过程也必然终止。届时如图(f)所示，全堆的堆序性必将恢复；而且，下滤乃至整个删除算法的时间复杂度也为 $O(\log n)$ ——同样，这从另一方面兑现了此前的承诺。

■ 实例

通过下滤变换实现删除操作的一个实例，如图10.6所示。同样地，图中上方和下方分别为完全堆的拓扑结构以及对应的线性存储结构。

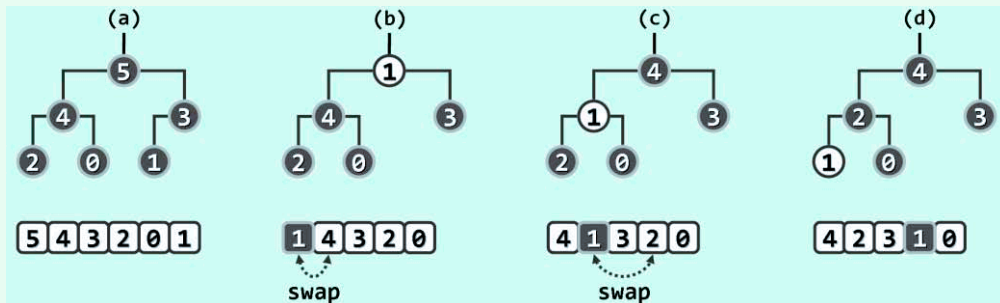


图10.6 完全二叉堆词条删除操作实例

从如图(a)所示由6个元素组成的完全堆中，现拟删除堆顶元素5。为此，首先如图(b)所示将该元素摘除，并将向量的末元素1转入首单元，权作堆顶。此后，1与其孩子节点均逆序。故如图(c)所示，在与其孩子中的大者4交换之后，1下降一层。此后，1与其新的孩子2依然逆序，故如图(d)所示经又一次交换后再下降一层。此时因1已抵达底层，删除操作完毕，算法成功终止。

■ 实现

以上调整在向量中的具体操作过程，可描述和实现如代码10.9所示。

```

1 //对向量前n个词条中的第i个实施下滤操作, i < n
2 template <typename T> Rank PQ_ComplHeap<T>::percolateDown(Rank n, Rank i) {
3     Rank j; //i及其(至多两个)孩子中, 堪为父者
4     while (i != (j = ProperParent(_elem, n, i))) //只要i非j, 则
5         { swap(_elem[i], _elem[j]); i = j; } //二者换位, 并继续考查下降后的i
6     return i; //返回下滤抵达的位置(亦i亦j)
7 }

```

代码10.9 完全二叉堆的下滤

这里为简化算法描述使用了宏ProperParent(), 其定义如288页代码10.3所示。

出于与上滤操作同样的考虑(习题[10-12]), 这里也可通过输入参数i, 灵活地指定起始位置。此前针对上滤操作所建议的改进方法, 有的也同样适用于下滤操作(习题[10-3]), 但有的却不再适用(习题[10-4])。

10.2.4 建堆

很多算法中输入词条都是成批给出, 故在初始化阶段往往需要解决一个共同问题: 给定一组词条, 高效地将它们组织成一个堆。这一过程也称作“建堆”(heapification)。本节就以完全二叉堆为例介绍相关的算法。当然, 以下算法同样也适用其它类型的堆。

■ 蛮力算法

乍看起来, 建堆似乎并不成其为一个问题。既然堆符合优先级队列ADT规范, 那么从空堆起